App Dev Project Report

1. Student Details:

Name: Saanjhi Varshney
Roll Number: 24f2003813
Email: 24f2003813@ds.study.iitm.ac.in

About Me: I am a student in the IIT Madras BS Degree Program with a strong interest in full-stack web development and software engineering. I enjoy designing scalable applications, solving real-world problems through technology, and continuously learning new tools and frameworks.

---

2. Project Details:

Problem Statement:
To develop a web-based application that simplifies trek management by allowing users to explore, book, and manage treks while enabling administrators to efficiently organize treks, guides, and bookings.

Approach:
The application was built using Flask for the backend and Vue.js for the frontend. It provides secure JWT-based authentication, trek booking and payment, booking history, badge management, and automated background tasks such as email reminders, monthly reports, and CSV exports using Celery and Redis.

---

3. AI/LLM Declaration: I used ChatGPT (GPT-5) to assist with understanding and debugging Celery and Redis integration, generating a small portion of the (YAML) documentation, and obtaining code suggestions for minor implementation details. The extent of AI/LLM usage is approximately 10–15%, primarily for guidance, debugging support, and documentation. All core application design, implementation, business logic, testing, and integration were completed independently.

---

4. Technologies and Frameworks Used:

Technology / Library	Purpose
Flask	Core backend web framework for developing REST APIs
Vue.js 3	Frontend framework for building the user interface
Vue Router	Client-side routing and navigation
Axios	HTTP client for communication between frontend and backend
Bootstrap 5	Responsive UI design and styling
SQLite	Lightweight relational database for storing application data
SQLAlchemy	Object Relational Mapper (ORM) for database operations
Flask-JWT-Extended	JWT-based user authentication and authorization
Redis	In-memory data store used for caching and Celery message broker
Flask-Caching	Improves application performance through caching
Celery	Executes asynchronous background tasks (emails, exports, reports)
Celery Beat	Schedules periodic background tasks

---

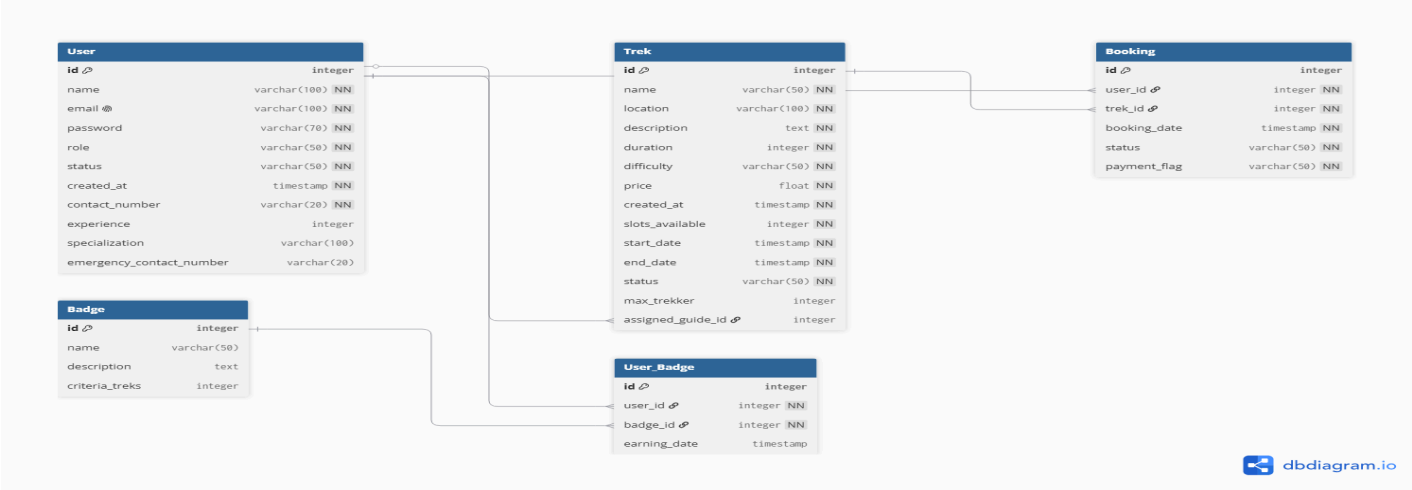
5. Database Schema / ER Diagram:

Tables:

- **User** — stores user account and profile details *(id, name, email, password, role, status, contact_number, experience, specialization, emergency_contact_number, created_at)*.
- **Trek** — stores trekking event details *(id, name, location, description, duration, difficulty, price, slots_available, start_date, end_date, status, max_trekker, assigned_guide_id, created_at)*.
- **Booking** — stores trek booking information *(id, user_id, trek_id, booking_date, status, payment_flag)*.
- **Badge** — stores achievement badge information *(id, name, description, criteria_treks)*.
- **User_Badge** — stores badges earned by users *(id, user_id, badge_id, earning_date)*.

Relationships:

- **One-to-Many → User → Trek** *(One guide can be assigned to many treks through `assigned_guide_id`.)*
- **One-to-Many → User → Booking** *(One user can have multiple bookings.)*
- **One-to-Many → Trek → Booking** *(One trek can have multiple bookings.)*
- **One-to-Many → User → User_Badge** *(One user can earn multiple badges.)*
- **One-to-Many → Badge → User_Badge** *(One badge can be awarded to multiple users.)*
- **Many-to-Many → User ↔ Trek** *(Implemented through the **Booking** table.)*



---

6. API Resource Endpoints:

Endpoint	Method	Description
`/api/auth/register`	POST	Register a new trekker account.
`/api/auth/login`	POST	Authenticate user and generate JWT token.
`/api/export-history`	POST	Export booking history and send it via email.
`/api/admin/dashboard`	GET	Retrieve admin dashboard summary and recent bookings.
`/api/admin/treks`	GET	View all treks with optional search.
`/api/admin/treks`	POST	Add a new trek.
`/api/admin/treks/{trek_id}`	PUT	Update trek details.
`/api/admin/treks/{trek_id}`	DELETE	Delete a trek.
`/api/admin/eligible_guides`	GET	Get staff members eligible to be assigned as guides.
`/api/admin/treks/{trek_id}/assign_guide`	PUT	Assign a guide to a trek.
`/api/admin/staff`	GET	View all staff members.
`/api/admin/staff`	POST	Add a new staff member.
`/api/admin/staff/{staff_id}`	PUT	Update staff information.
`/api/admin/staff/{staff_id}`	DELETE	Delete a staff member.
`/api/admin/users`	GET	View all registered trekkers.
`/api/admin/bookings`	GET	View all bookings with search support.
`/api/admin/bookings/{booking_id}/status`	PUT	Update booking status.
`/api/admin/bookings/{booking_id}/status`	DELETE	Delete a booking.
`/api/staff/dashboard`	GET	View staff dashboard overview.
`/api/staff/trek`	GET	View assigned trek details.
`'api/staff/treks/{trek_id}/slots_capacity`	PUT	Update trek capacity.

`/api/staff/treks/{trek_id}/status`	PUT	Update trek status.
`/api/staff/participants`	GET	View participants of assigned treks.
`/api/staff/profile`	GET	View staff profile.
`/api/staff/profile`	PUT	Update staff profile.
`/api/user/dashboard`	GET	View available treks and current bookings.
`/api/user/history`	GET	View completed and cancelled booking history.
`/api/user/booking_cancel/{booking_id}`	PUT	Cancel a booked trek.
`/api/bookings/{booking_id}/pay`	PUT	Update booking payment status.
`/api/user/edit_profile`	GET	View trekker profile.
`/api/user/edit_profile`	PUT	Update trekker profile.
`/api/trekker/treks`	GET	View all open treks.
`/api/trekker/treks/{trek_id}`	GET	View details of a specific trek.
`/api/trekker/treks/{trek_id}/book`	POST	Book a trek.
`/api/user/badges`	GET	View earned badges and progress.

YAML API Definition File: api.yaml

---

7. Architecture and Features (optional)

Architecture Overview:

- app.py – Main Flask application and API entry point
- models.py – SQLAlchemy models for users, treks, bookings, and badges
- extensions.py – Initialization of Flask extensions (SQLAlchemy, JWT, Redis Cache, Celery)
- tasks.py – Celery background tasks for reminders, reports, and exports
- Vue.js Components – Frontend pages for admin and trekker dashboards
- SQLite Database – Stores users, treks, bookings, and badges

Implemented Features:

- User registration and JWT-based login
- Role-based access for Admin and Trekker
- Trek creation, update, and management
- Trek booking and payment status management
- Trek booking history and profile management
- Badge awarding based on completed treks
- Dashboard with trek and booking statistics
- Secure password hashing and authentication

Additional Features:

- Automated email reminders using Celery and Redis
- Monthly activity reports sent via email
- Export booking history as CSV
- Scheduled background tasks using Celery Beat

---

8. Video Presentation

Drive Link:https://www.loom.com/share/b56c53eed4d847ff8fbae52002959654